

Numpy notes

Phase 1

1. How to create an array.

Creating an array is one of the most fundamental functions of Numpy - here is a normal python list initialization -

```
py_list = [1,2,3]
```

and Now, this is the main Numpy array initialization -

```
np_array = np.array([1,2,3])
```

| curly braces and then square bracket - for NUMPY array.

2. Multiplication of an array in numpy:

as simple as it is - pretty intuitive , here we go :

the earlier numpy array was given the name - np_array.

therefore to print multiplication of that, simply write :

```
print("Python array mulitplication : ", np_array * 2 )
```

this is the main thing for multiplication - np_array * 2 - as straightforward as it gets

3. zeros, ones & fulls:

There are 3 types of arrays - matrices which are matrices of all zeros, all ones and all full of one number.

a) Zeros :

Initialization : **zeros = np.zeros((3,4))**

```
print - print("zeros array: \n " , zeros)
```

| the (3,4) represent the columns and rows respectively

```
zeros array:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

```
b) Ones :
ones = np.ones((4,4))
print("ones array: \n ", ones)
```

| the (4,4) represent the columns and rows of the array

```
ones array:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
c) Full :
full = np.full((3,3), 7)
print("Full Array of 7: \n", full)
```

```
full array:
[[7 7 7]
 [7 7 7]
 [7 7 7]]
```

d) Random Array **(IMPORTANT)**

```
random = np.random.random((2,3))
print("Random Array : \n ", random)
```

Random.random - it is because the first random is a a class in itself, and if anything is generated randomly - it is generated between 0 and 1 only.

```
random array:  
[[0.13828463 0.23698081 0.95318988]  
 [0.60143905 0.79916152 0.66809963]]
```

e) Sequence array

It is exclusive of the last number , therefore 10 is not there - only 8 , the 2 is the number to skip - 0,2,4,6,8.

```
sequence = np.arange(0,10, 2)  
print("sequence array: \n", sequence)
```

```
sequence array:  
[0 2 4 6 8]
```

4) Vector, Matrix and Tensor

A) Vector - simple stuff

```
vector = np.array([1,2,3,4])  
print("Vector: ", vector)
```

the vector's initializatoin will have ([])

```
Vector: [1 2 3 4]
```

B) Matrix - next level - 2 dimension

```
matrix = np.array([[1,2,3],  
                  [4,5,6]])  
print("Matrix: ", matrix)
```

the matrix has more than what vector had, it has this ([[]]) - double square brackets

```
Matrix:  [[1 2 3]  
         [4 5 6]]
```

C) Tensor - max level - crazy stuff hai ye

```
tensor = np.array([[[1,2], [3,4]],  
                  [[5,6], [7,8]]])  
print("Tensor: ", tensor)
```

```
Tensor:  [[[1 2]  
          [3 4]  
  
          [5 6]  
          [7 8]]]
```

5) Array Properties

```
arr = np.array([[1,2,3],  
               [4,5,6]])
```

```
print("Array", arr)
print("Array shape", arr.shape)
print("Array Dimension", arr.ndim)
print("Array size", arr.size)
print("Array data type", arr.dtype)
```

Here are different information which can be taken from one single numpy array :

- 1) Array shape
- 2) Dimension
- 3) Size
- 4) Data type ;

The following is the result for the above code snippet

```
Array:  [[1 2 3]
         [4 5 6]]
Shape  (2, 3)
Dimension  2
Size  6
Dtype  int64
```

6) Array Reshaping

```
arr = np.arange(12)
print("OG Array", arr)

reshaped = arr.reshape((3,4))
print("\n Reshaped Array", reshaped)

flattened = reshaped.flatten
```

```

print("\n Flattened Array", flattened)

#ravel - ravel returns view instead of copy
ravel = reshaped.ravel()
print("\n Raveled Array", ravel)

#transpose - jo 11th-12th mei karte the
transposed = reshaped.T
print(" \n Transposed Array: ", transposed)

```

This is the result for the above code snippet :

```

Original Array [ 0  1  2  3  4  5  6  7  8  9 10 11]

Reshaped Array  [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

Flattened array : <built-in method flatten of numpy.ndarray object at 0x1086abc90>

Raveled array : [ 0  1  2  3  4  5  6  7  8  9 10 11]

Transposed array : [[ 0  4  8]
 [ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]]

```

Phase 2

1) Numpy Array operations

```

arr = np.array([1,2,3,4,5,6,7,8,9,10])
print("Basic slicing: ", arr[2:7])
print("Basic slicing returns: ", arr[3:9])

```

```
print("with step: ", arr[1:8:2])
print("Negative indexing: ", arr[-3])
```

the result :

```
Basic Slicing:  [3 4 5 6 7]
Basic Slicing returns:  [4 5 6 7 8 9]
with step:  [2 4 6 8]
Negative indexing:  8
```

```
arr_2d = np.array([[1,2,3],
                   [4,5,6],
                   [7,8,9]])

#here starts specific stuff
print("Specific Element: ", arr_2d[1,2]) --> # the [1,2] means 1st row and 2nd column

# but remember -
1,2,3 is the 0th row, 4,5,6 is the 1st row, 7,8,9 is the 2nd row - same goes for columns
print("Entire Row : ", arr_2d[1]) --> 1st row therefore 4,5,6 will be returned
print("Entire Column: ", arr_2d[:, 1]) --> 1st column therefore 2 5 8 will be returned
```

the result :

```
specific element 6
Entire Row [4 5 6]
Entire column [2 5 8]
```

6 because 1st row - 4,5,6 and 2nd column so 6 is the element at [1,2]
row is 4 5 6 because 1st row was written (0, 1, 2) is the way of these matrices
not 1,2,3
likewise - column is 2 5 8 for 1st column

2) SORTING

```
unsorted = ([4,8,2,7,5,1,9,3,6])
print("Sorted Array: ", np.sort(unsorted))

array_2d_unsorted = np.array([[3,1], [1,2], [2,3]])
print("Sorted 2d array by column", np.sort(array_2d_unsorted), axis = 0)
```

the result :

```
Sorted array : [1 2 3 4 5 6 7 8 9]
Sorted 2d array by column
[[1 1]
 [2 2]
 [3 3]]
```

3) Filter

```
numbers = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
even_number = numbers[numbers % 2 == 0]
print("Even numbers", even_number)
```

```
Even numbers: [ 2  4  6  8 10]
```

Filter with mask

```
mask = numbers > 5
print("numbers greater than 5 : \n", numbers[mask])
```

```
Numbers greater than 5 :
[ 6  7  8  9 10]
```

Fancy indexing v/s np.where()

```
indices = [0,2,4]
print(numbers[indices])
```

```
where_result = np.where(numbers > 5)
print(where_result)
print("NP where", numbers[where_result])
```

```
[1 3 5]
(array([5, 6, 7, 8, 9]),)
NP where [ 6  7  8  9 10]
```